

Historia del Proyecto: Merci Boilerplate

Bitacora | Vol. 1

mercedev.es — 2026-04-25

Historia del Proyecto: Merci Boilerplate

Registro técnico estructurado — Vol. I

Formato: Contexto · Hecho · Detalle técnico · Motivo / criterio · Fuentes / Bibliografía

2026-04-12 — Fase 1: Infraestructura base, Merci Audit y primer commit

Contexto: El proyecto parte de cero bajo las directrices del archivo `instrucciones.md`, que establece tres pilares irrenunciables: rendimiento, seguridad *shift-left* y valor pedagógico. Antes de escribir una sola línea de HTML o CSS, era necesario definir cómo se organizaría el código, cómo se auditaría y cómo se versionaría. Sin esta base, cualquier avance posterior sería frágil e irreproducible.

Hecho: - Se definió la estructura de carpetas del repositorio: `public/` (sitio servido), `docs/` (documentación), `scripts/merci/` (automatización), `laboratorio/` (experimentos), `assets/` (multimedia optimizado) y `.assets-raw/` (originales sin procesar, excluidos de Git). - Las carpetas vacías se versionaron con archivos `.gitkeep` para que un `git clone` conserve el esqueleto completo del proyecto. - Se implementó `scripts/merci/merci-audit.py`, auditor estático escrito íntegramente con la biblioteca estándar de Python (sin dependencias `pip`), capaz de detectar secretos expuestos, errores de sintaxis en `.py` y `.json`, patrones peligrosos en JavaScript (`eval`, `new Function`) y reglas mínimas de SEO técnico en HTML. - Se creó `scripts/merci/pre-commit`, un shell que invoca el auditor en modo `--git-staged` (solo

sobre los archivos que van al commit) y se enlazó simbólicamente a `.git/hooks/pre-commit`. - Se configuró `.gitignore` para excluir entornos virtuales, cachés y artefactos de compilación. - Se realizó el commit inicial en la rama `main` con el mensaje `chore: commit inicial – Fase 1 (estructura, Merci Audit, directrices)`.

Detalle técnico: El auditor tiene tres modos de ejecución según el contexto:

```
# Auditoría sobre todo el árbol del repositorio
python3 scripts/merci/merci-audit.py

# Solo archivos en el índice de Git (modo hook de pre-commit)
python3 scripts/merci/merci-audit.py --git-staged

# Exigir bloque JSON-LD en archivos HTML (para endurecimiento del CI)
python3 scripts/merci/merci-audit.py --strict-json-ld
```

El hook devuelve un código de salida `1` ante cualquier hallazgo de tipo `ERROR`, lo que Git interpreta como fallo y bloquea el commit. Los hallazgos de tipo `WARN` son informativos y no detienen la operación. El enlace simbólico al hook no viaja con el repositorio al hacer `clone`; debe recrearse en cada máquina nueva (o documentarse en un script de *bootstrap*).

Motivo / criterio: Automatizar las comprobaciones de calidad y seguridad antes de integrar cambios es la definición práctica de *Shift-Left*. Usar solo la biblioteca estándar de Python en esta fase garantiza que el auditor funcione en cualquier máquina con Python instalado, sin necesidad de ejecutar `pip install` previamente. El modo `--git-staged` evita auditar el árbol completo en cada commit, manteniendo el flujo de trabajo ágil.

Fuentes / Bibliografía: - Asistencia mediante IA para el diseño de la arquitectura de carpetas y la estrategia de auditoría *shift-left*. - Documentación oficial de Git sobre el mecanismo de hooks de `pre-commit`. - Documentación oficial de Python sobre el módulo `subprocess` para la integración del auditor.

2026-04-12 — Política de gestión de assets brutos y apertura del repositorio

Contexto: Al planificar el flujo de trabajo con multimedia (imágenes, vídeos de evidencias), surgió el riesgo de que archivos pesados como PSD, RAW o grabaciones de pantalla acabaran versionados en GitHub por error, inflando el repositorio y bloqueando futuros `clone`. Paralelamente, se preparaba la apertura pública del repositorio y era necesario establecer el tono correcto de la documentación pública.

Hecho: - Se configuró `.gitignore` para ignorar el contenido de `.assets-raw/*` manteniendo únicamente el `.gitkeep` del directorio, mediante el patrón de negación `!.assets-raw/.gitkeep`. - Se añadió la regla 7 a `instrucciones.md`: los recordatorios al autor deben mantenerse fuera del repositorio; el texto versionado en Git debe estar redactado en tono neutro y ser útil para visitantes o colaboradores externos. - Se añadió la licencia MIT al repositorio y se redactó el `README.md` en tono técnico, destacando conceptos como «Shift-Left», «aislamiento de sistemas» y «trazabilidad».

Detalle técnico: El patrón de exclusión en `.gitignore` funciona mediante dos líneas complementarias:

```
.assets-raw/*
!.assets-raw/.gitkeep
```

La primera línea excluye todo el contenido del directorio. La segunda, con el prefijo `!` (negación), reintroduce el archivo `.gitkeep` para que el directorio vacío sea rastreable por Git. Los assets optimizados (WebP, variantes responsivas) sí se versionan en `assets/`, ya que son artefactos de compilación ligeros y reproducibles.

Motivo / criterio: Un repositorio ligero y reproducible. Los archivos originales (brutos) tienen su lugar en el disco local, en un NAS o en almacenamiento externo, no en el control de versiones. Versionar solo los artefactos finales optimizados garantiza clones rápidos y evita que el repositorio supere las cuotas de GitHub.

Además, un `README` con lenguaje de ingeniería transmite la madurez técnica del proyecto a cualquier visitante o reclutador.

Fuentes / Bibliografía: - Documentación oficial de Git sobre patrones de negación en `.gitignore` . - Licencia MIT: <https://opensource.org/licenses/MIT>.

2026-04-14 — Fase 2: HTML semántico, JSON-LD, sitemap y automatización de metadatos

Contexto: Con la infraestructura de auditoría lista, el siguiente paso era construir el primer documento HTML público. Pero antes de escribir el contenido, era necesario que el archivo fuera técnicamente correcto desde el primer commit: semántica HTML5, datos estructurados para buscadores (JSON-LD) y metadatos de indexación (`robots.txt` , `sitemap.xml`). El reto adicional era que la fecha del sitemap debía mantenerse sincronizada con los cambios de contenido sin intervención manual.

Hecho: - Se creó `public/index.html` con semántica HTML5 completa, bloque JSON-LD de `schema.org` , atributos `lang` , `charset` y metadatos `Open Graph` . - Se añadieron las etiquetas `aria-label` al elemento `<nav>` y se corrigió la jerarquía de encabezados para garantizar una secuencia `h1 > h2 > h3` sin saltos. - Se crearon `public/robots.txt` (rastreo total permitido, enlace al sitemap) y `public/sitemap.xml` (URL canónica raíz, prioridad máxima). - Se implementó `scripts/merci/merci-sitemap.py` , que actualiza automáticamente la fecha `<lastmod>` del sitemap. - Se integró `merci-sitemap.py` en el hook `pre-commit` : si hay archivos staged en `public/` , el hook ejecuta el actualizador de sitemap y añade el archivo resultante al commit en curso (`git add public/sitemap.xml`). - El archivo `public/index.html` superó la auditoría con `merci-audit.py --strict-json-ld` sin errores ni advertencias.

Detalle técnico: `merci-sitemap.py` utiliza el módulo `datetime` para obtener la fecha del sistema y `re.sub` para sustituir el valor de `<lastmod>` en el XML sin necesidad de parsers pesados:

```
import re
from datetime import date

new_date = date.today().isoformat()
content = re.sub(r'<lastmod>.*?</lastmod>', f'<lastmod>{new_date}</lastmod>', content)
```

La integración en el hook de `pre-commit` funciona así: si `git diff --cached --name-only` devuelve alguna ruta bajo `public/`, se ejecuta el script y se ejecuta `git add public/sitemap.xml` para incluir el sitemap actualizado en el mismo commit atómico, sin pasos manuales adicionales.

Motivo / criterio: Garantizar que el sitio sea indexable y cumpla los estándares de SEO técnico desde la primera línea de código es la aplicación directa del principio *Shift-Left* al posicionamiento web. La automatización de `<lastmod>` evita una deuda técnica silenciosa: un sitemap con fechas desactualizadas confunde a los rastreadores de Google y penaliza la frecuencia de reindexación. Resolver esto en el hook elimina la posibilidad del error humano.

Fuentes / Bibliografía: - Documentación de `schema.org` para el bloque JSON-LD: <https://schema.org>. - Guía de Google sobre sitemaps XML: <https://developers.google.com/search/docs/crawling-indexing/sitemaps/build-sitemap>. - Especificación WAI-ARIA de W3C: <https://www.w3.org/TR/wai-aria>.

2026-04-15 — Fase 3: Sistema de estilos SASS 7-1 y compilador autónomo

Contexto: El núcleo estático necesitaba un sistema de diseño escalable. La opción evidente era usar SASS con arquitectura modular, pero la herramienta estándar para compilarlo (Dart Sass) requiere Node.js globalmente instalado, lo que contravenía la filosofía de «cero dependencias externas en el sistema operativo anfitrión». La librería Python alternativa (`libsass`) no soportaba las directivas modulares modernas (`@use` , `@forward` , `_index.scss`), imprescindibles para una arquitectura desacoplada.

Hecho: - Se diseñó la arquitectura SASS 7-1 en `src/scss/` con un punto de entrada único (`main.scss`) que importa únicamente los índices de cada subdirectorio. - Se implementó `scripts/merci/merci-styles.py` : un script que descarga automáticamente el binario standalone de Dart Sass para Linux, lo extrae en `scripts/merci/bin/dart-sass/` y lanza la compilación de `src/scss/main.scss` a `public/css/main.css` mediante `subprocess` . - Se implementó `scripts/merci/merci-watcher.py` : un vigilante que monitoriza cambios en `src/scss/` usando `path.stat().st_mtime` e invoca a `merci-styles.py` en tiempo real durante las sesiones de diseño. - Se añadieron alias de terminal en `~/.zshrc` para ambas herramientas (`merci-styles` , `merci-watch`). - Se refactorizó `public/index.html` para adoptar la nomenclatura BEM (*Block, Element, Modifier*) de forma coherente. - Se migró el uso de funciones globales de color de Dart Sass (`scale-color`) al módulo integrado moderno (`color.scale`), eliminando todos los *Deprecation Warnings* del compilador.

Detalle técnico: La arquitectura SASS 7-1 depende de una cadena de importación sin ambigüedades. Un componente (ej. `_section.scss`) debe estar declarado con `@forward` en su índice local (`components/_index.scss`), y ese índice debe ser importado por el punto de entrada principal (`main.scss`). Si cualquier eslabón de esta cadena falta, el compilador ignora silenciosamente el componente y sus reglas no aparecen en el CSS de salida — un error que se manifiesta como «el estilo no se aplica» sin ningún mensaje de error explícito.

El binario de Dart Sass se almacena localmente y se excluye de Git mediante `.gitignore` , pero el script lo descarga automáticamente si no existe, garantizando que el compilador esté siempre disponible sin intervención manual.

La migración de funciones de color sigue este patrón:

```
// ❌ Deprecated (Dart Sass < 3.0)
color: scale-color($color-primary, $lightness: -20%);

// ✅ Moderno
@use 'sass:color';
color: color.scale($color-primary, $lightness: -20%);
```

Motivo / criterio: Usar un binario standalone de Dart Sass permite compilar SASS moderno con soporte completo de módulos (`@use` / `@forward`) sin instalar Node.js ni NPM en el sistema operativo. Esto preserva el principio de austeridad tecnológica: el proyecto debe poder construirse en cualquier máquina Linux con solo Python instalado. Atender las deprecaciones del compilador desde el inicio evita que el proyecto acumule deuda técnica que se vuelva bloqueante en futuras versiones de Dart Sass.

Fuentes / Bibliografía: - Documentación oficial de Dart Sass sobre módulos `@use` y `@forward` : <https://sass-lang.com/documentation/at-rules/use>. - Repositorio de releases de Dart Sass (binario standalone): <https://github.com/sass/dart-sass/releases>. - Guía de arquitectura SASS 7-1: <https://sass-guidelin.es/#the-7-1-pattern>.

2026-04-15 — Implementación del linter de acrónimos y política de expansión

Contexto: El proyecto tiene una vocación pedagógica explícita: cualquier término técnico debe ser comprensible para alguien que se acerque por primera vez. Sin embargo, exigir la expansión de todos los acrónimos en todo momento (ej. escribir «CSP (Content Security Policy — Política de Seguridad de Contenidos)» cada vez que aparece la sigla) generaba una fricción real en el flujo de escritura de la bitácora y la documentación.

Hecho: - Se implementó la función `audit_md_acronyms` en `merci-audit.py`, con una lista de vigilancia (*watchlist*) de los acrónimos técnicos más críticos del proyecto. - La función detecta si un acrónimo aparece sin su expansión y emite un `WARN MD_ACRONYM` indicando la línea exacta, sin bloquear el commit. - Se añadió la función `get_global_acronym_count`, que escanea todos los archivos `.md` del repositorio y cuenta las apariciones del acrónimo. Si supera las 3 apariciones globales, el término se considera consolidado y la advertencia se omite. - Se implementó una caché interna (`GLOBAL_ACRONYM_COUNTS`) para evitar leer el disco repetidas veces por sesión. - La validación del linter se confirmó en

producción: al ejecutar un commit con «CMS» sin expandir, el auditor emitió el aviso correctamente sin bloquear la operación.

Detalle técnico: La función utiliza expresiones regulares para detectar si un acrónimo de la *watchlist* está presente en un archivo `.md` sin el patrón de expansión `ACRÓNIMO (...)`. El clasificador como `WARN` (y no `ERROR`) es deliberado: los falsos positivos en análisis de texto son inevitables con una expresión regular genérica, y bloquear commits por un falso positivo destruiría la experiencia de desarrollo. La advertencia informa sin obstaculizar.

```
# Patrón de detección de acrónimo sin expansión
pattern_unexpanded = re.compile(r'\b' + acronym + r'\b(?:\s*\(\))')
# Patrón de verificación de expansión existente
pattern_expanded = re.compile(r'\b' + acronym + r'\s*\(\)')
```

Motivo / criterio: Equilibrar la claridad pedagógica con la fluidez del proceso. Forzar la expansión en la primera aparición de un término garantiza que un lector nuevo pueda entender el documento. Eximir del requisito cuando el término ya es de dominio público en el repositorio (más de 3 apariciones) reduce el tedio sin sacrificar la intención original. La automatización parcial mediante *watchlist* es más fiable que una expresión regular genérica para mayúsculas.

Fuentes / Bibliografía: - Asistencia mediante IA para el diseño de la estrategia de conteo global con caché en memoria. - Documentación oficial de Python sobre el módulo `re` para expresiones regulares.

2026-04-15 — Fase 5: Content Security Policy y Hardening del núcleo estático

Contexto: Con el núcleo estático construido y auditado, era el momento de aplicar la primera capa de seguridad activa. El vector de ataque más común en páginas web es el XSS (Cross-Site Scripting): la inyección de scripts maliciosos en el DOM del navegador. Una Content Security Policy (CSP — Política de Seguridad de Contenidos) estricta es el mecanismo estándar de la industria para mitigar este

riesgo. Sin embargo, una CSP mal configurada puede bloquear recursos legítimos y romper el sitio.

Hecho: - Se añadió la directiva CSP en el `<head>` de `public/index.html` :
`default-src 'self'` (todos los recursos deben provenir del mismo dominio),
`object-src 'none'` (sin plugins embebidos) y `base-uri 'self'` (prevención de inyección de base URL). - Se validó la CSP desplegando un servidor local de pruebas (`python3 -m http.server 8000 -d public/`) y revisando la consola del navegador: cero errores de violación de CSP. - Se migró posteriormente la CSP de la etiqueta `<meta>` a una cabecera HTTP real en Nginx (más efectiva contra ataques XSS según las directrices de Google). - Se añadieron las cabeceras `Strict-Transport-Security` (HSTS con directiva `preload`), `Cross-Origin-Opener-Policy` (COOP), `Cross-Origin-Embedder-Policy` (COEP), `Referrer-Policy` y `X-Content-Type-Options` en el bloque `server` de Nginx.

Detalle técnico: La implementación via cabecera HTTP es el método de *enforcement* correcto. Una CSP definida como etiqueta `<meta>` no tiene efecto contra todos los vectores de ataque (en particular, no puede bloquear iframes ni ciertas directivas de navegación). La cabecera HTTP, en cambio, es aplicada por el servidor antes de que el navegador procese ningún byte del HTML.

La directiva `preload` en HSTS inscribe el dominio en las listas maestras de los navegadores (Chrome, Firefox, Edge), garantizando que las conexiones sean HTTPS incluso en la primera visita — eliminando el milisegundo de vulnerabilidad inicial. La inscripción en las listas de preload es irreversible a corto plazo, por lo que se documentó como una decisión deliberada y con pleno conocimiento de sus implicaciones.

Se documentó explícitamente la decisión de **no implementar** la directiva `require-trusted-types-for` : activarla fracturaría la operatividad de WordPress, sus plugins y el editor de bloques (Gutenberg), cuyo código base no es compatible nativamente con esta API. Se clasificó como *Deuda Técnica conocida y asumida*, derivada del uso de un CMS maduro.

Motivo / criterio: Aplicar el principio de seguridad *Shift-Left*: implementar la CSP desde la fase de desarrollo local, no como parche post-despliegue. Una CSP estricta mitiga vectores de ataque XSS, Clickjacking, MIME-sniffing y ataques de

canal lateral (Spectre). Validarla en local antes de desplegar a producción garantiza que la política no degrada la experiencia de usuario (UX) ni bloquea recursos legítimos.

Fuentes / Bibliografía: - Documentación de MDN Web Docs sobre Content Security Policy: <https://developer.mozilla.org/es/docs/Web/HTTP/CSP>. - Guía de Google sobre cabeceras de seguridad: <https://web.dev/security-headers>. - HSTS Preload List: <https://hstspreload.org>.

2026-04-15 — Fase 4.1: Arquitectura de aislamiento de WordPress y Child Theme ultraligero

Contexto: El proyecto necesitaba integrar WordPress para gestionar el blog y el catálogo de WooCommerce, pero sin sacrificar el rendimiento ni la seguridad del núcleo estático. WordPress, por defecto, inyecta docenas de scripts, estilos en línea y dependencias que degradan las métricas de Core Web Vitals. La solución no era evitar WordPress, sino aislarlo completamente.

Hecho: - Se diseñó y documentó la arquitectura de aislamiento en `docs/integracion-wordpress.md`: el núcleo estático vive en `public/`, WordPress en un directorio hermano completamente separado (`/var/www/wordpress/`), y Nginx actúa como proxy inverso enrutando `/blog` al CMS y el resto a archivos estáticos. - Se creó el directorio `src/wp-theme/merci-theme/` con el manifiesto `style.css` (solo la cabecera de comentarios requerida por WordPress). - Se implementó `functions.php` como «escudo de rendimiento»: desencolado de scripts de emojis, bloqueo de `global-styles`, `classic-theme-styles` y bloques de Gutenberg; desactivación de XML-RPC, ofuscación de errores de autenticación y ocultación de la versión de WordPress. - Se implementó `index.php` con el bucle de WordPress usando semántica HTML5 y clases BEM, prescindiendo de la fragmentación tradicional (`get_header()`, `get_footer()`). - Se configuró WooCommerce en **modo catálogo**: se añadió `add_theme_support('woocommerce')`, se bloquearon los botones de compra y se desencoló el script `wc-cart-fragments` (responsable de una petición POST innecesaria en cada carga de página). - Se implementó un filtro global

`merci_defer_js_frontend` que inyecta el atributo `defer` en todas las etiquetas `<script>` del frontend, garantizando que el parseo de HTML nunca sea interrumpido por JavaScript.

Detalle técnico: El enlace simbólico es el mecanismo clave de la arquitectura de aislamiento: conecta el código del tema (gobernado por Git) con el directorio de temas del CMS (externo al repositorio), de modo que un `git pull` actualiza el diseño automáticamente sin copiar archivos.

```
# Enlace simbólico del Child Theme en entorno local
ln -s /home/usuario/PROYECTO_mercedev.es/src/wp-theme/merci-theme \
    /var/www/wordpress/wp-content/themes/merci-theme
```

El script `wc-cart-fragments` invoca una petición `POST` a `/?wc-ajax=get_refreshed_fragments` en cada carga de página. Al ser un `POST` que verifica sesiones y bases de datos mediante PHP, esquivando todas las capas de caché (Varnish, Nginx FastCGI), elevando el consumo de CPU (Central Processing Unit — Unidad Central de Procesamiento) y el TTFB (Time to First Byte — Tiempo hasta el Primer Byte). En modo catálogo, este script aporta cero funcionalidad a costa de destruir las métricas de Core Web Vitals.

La seguridad del CMS se endurece aplicando el principio de mínimo privilegio: directorios a `755`, archivos a `644` y `chmod 600` estricto para `wp-config.php`, con la propiedad asignada a `www-data:www-data`.

Motivo / criterio: Aislar los vectores de ataque del CMS del núcleo estático. Si WordPress es vulnerado (plugins desactualizados, brecha en un proveedor), el frontend estático queda completamente ileso porque reside en un directorio sin acceso de escritura por parte del proceso PHP. Además, servir los assets estáticos directamente con Nginx (sin pasar por PHP) garantiza que las métricas de rendimiento del núcleo no sean afectadas por la carga del CMS.

Fuentes / Bibliografía: - Documentación oficial de WordPress sobre el sistema de hooks y el `functions.php` de Child Themes: <https://developer.wordpress.org/themes/advanced-topics/child-themes>. - Documentación de WooCommerce sobre el filtro `add_theme_support`: <https://woocommerce.com/document/declaring->

[woocommerce-support-in-themes](#). - Asistencia mediante IA para el análisis del impacto de `wc-cart-fragments` en Core Web Vitals.

2026-04-15 — Sistema Merci: automatización de commits impulsada por la bitácora

Contexto: La bitácora del proyecto (este mismo documento) se redactaba manualmente en Markdown, pero su conexión con el historial de Git era manual y propensa a errores: era posible modificar código sin actualizar la bitácora, o actualizar la bitácora sin comitear el código correspondiente. Esta desincronización rompe la trazabilidad del proyecto y hace que el historial de Git pierda su valor como registro de decisiones.

Hecho: - Se diseñó y desarrolló `scripts/merci/merci-commit.py`: un script que extrae automáticamente el título de la última entrada de la bitácora y lo usa como mensaje del commit de Git. - El script ejecuta `git add .` desde la raíz del repositorio antes del commit (con `cwd=REPO_ROOT`), garantizando que todo el código modificado viaja en el mismo commit atómico que su justificación. - Se añadió una salvaguarda: el script detecta si la bitácora no ha sido modificada (`git diff --quiet HEAD <ruta_bitácora>`) y alerta al usuario antes de proceder, requiriendo confirmación explícita. - Se añadió soporte para commits menores sin entrada en la bitácora: si hay cambios de código pero no en la bitácora, el script solicita un mensaje manual por terminal (`input()`) en lugar de bloquearse o fallar silenciosamente. - Se añadió `merci-commit` como alias en `~/.zshrc`.

Detalle técnico: El extractor de título analiza la última entrada de la bitácora buscando el patrón de encabezado Markdown (`## FECHA – Título`) mediante una expresión regular. El resultado se usa directamente como mensaje del commit, garantizando que el historial de Git refleje siempre el lenguaje de la bitácora:

```
pattern = re.compile(r'^##\s+\d{4}-\d{2}-\d{2}\s+--\s+(.+)$',
re.MULTILINE)
```

```
matches = pattern.findall(bitacora_content)
title = matches[-1].strip() # Última entrada
```

La verificación previa de cambios en Git usa `git status --porcelain` para detectar el estado real de los archivos, diferenciando entre «sin cambios» (abortar), «cambios con bitácora actualizada» (flujo estándar) y «cambios sin bitácora actualizada» (flujo de parche menor).

Motivo / criterio: Mantener un historial de Git semántico y atómico: cada commit contiene exactamente un cambio lógico coherente, con su justificación documentada. Esto convierte el `git log` en un registro de decisiones técnicas, no en una lista de mensajes crípticos. La salvaguarda contra commits sin bitácora refuerza la disciplina de «documentación primero» sin añadir fricción paralizante al flujo de trabajo.

Fuentes / Bibliografía: - Especificación de Conventional Commits para la nomenclatura de mensajes: <https://www.conventionalcommits.org>. - Documentación oficial de Git sobre `git status --porcelain`. - Asistencia mediante IA para el diseño del flujo de parche menor con `input()` de Python.

2026-04-16 — Resolución de errores de infraestructura y simlinks en el Child Theme

Contexto: Durante el desarrollo local de la integración con WordPress, se produjeron dos incidentes de sistema de archivos relacionados con los enlaces simbólicos del Child Theme: un bucle infinito de directorios (symlink loop) y un archivo HTML residual dentro del directorio del tema. Estos problemas no afectaban al funcionamiento del CMS, pero amenazaban con colgar el indexador del editor de código (VS Code) y corromper el árbol de Git.

Hecho: - Se identificó un bucle de enlaces simbólicos dentro de `src/wp-theme/merci-theme/`: un enlace apuntaba a su propio directorio padre, creando una referencia circular con apariencia de subdirectorios infinitos. - Se eliminaron los enlaces erróneos con `rm -rf src/wp-theme/merci-theme/*/` y

`find -type l -delete` . - Al intentar recuperar los archivos con `git restore` , el bucle reapareció porque el enlace simbólico erróneo estaba registrado en un commit anterior del historial de Git. - Se realizó una «cirugía manual»: extracción temporal de los tres archivos críticos (`index.php` , `functions.php` , `style.css`), destrucción y recreación limpia del directorio, y restitución de los archivos para forzar la actualización del índice de Git. - Se eliminó el archivo `src/wp-theme/merci-theme/index.html` residual (contenido temporal de la página de Contacto que había quedado en el directorio equivocado) mediante `git rm` .

Detalle técnico: Un *symlink loop* ocurre cuando un enlace simbólico apunta a una ruta que lo contiene como ancestro, creando una referencia circular. El tamaño real en disco es cero bytes, pero los indexadores (VS Code, Git) que intentan recorrer el árbol de directorios pueden entrar en un bucle infinito de resolución de rutas.

La secuencia de recuperación que rompe definitivamente el bucle a nivel de sistema de archivos y de Git:

```
# 1. Preservar los archivos válidos
mv src/wp-theme/merci-theme/index.php /tmp/
mv src/wp-theme/merci-theme/functions.php /tmp/
mv src/wp-theme/merci-theme/style.css /tmp/

# 2. Destruir el directorio corrupto
rm -rf src/wp-theme/merci-theme/

# 3. Recrear limpio
mkdir -p src/wp-theme/merci-theme/

# 4. Restituir los archivos
mv /tmp/index.php /tmp/functions.php /tmp/style.css src/wp-theme/merci-theme/
```

El commit posterior sobrescribe el estado del árbol en Git, purgando permanentemente la referencia al enlace simbólico fantasma del historial.

Motivo / criterio: `git restore` recupera fielmente el historial, incluyendo los errores que estén registrados en commits anteriores. La cirugía manual de directorios es la intervención más segura y pragmática para romper dependencias

circulares en el sistema de archivos antes de conciliar el estado limpio con el control de versiones. El directorio `merci-theme` solo debe contener la tríada de archivos planos de un Child Theme; cualquier directorio anidado es, por definición de esta arquitectura, un residuo.

Fuentes / Bibliografía: - Documentación del sistema de archivos Linux sobre el comportamiento de los enlaces simbólicos y las referencias circulares. - Asistencia mediante IA para el diseño de la secuencia de recuperación sin pérdida de datos.

2026-04-16 — Pivote estratégico: de web personal a Merci Boilerplate

Contexto: A mitad del desarrollo se tomó una decisión arquitectónica de alto nivel: el valor real del proyecto no residía en el sitio web personal de `mercedev.es`, sino en la infraestructura híbrida, el sistema de automatización (Sistema Merci) y las prácticas DevSecOps integradas. Esta infraestructura, si se abstraía correctamente, podía servir como base reutilizable para múltiples proyectos web futuros.

Hecho: - Se pivotó formalmente el objetivo del proyecto: de web personal (`mercedev.es`) a plantilla de desarrollo reutilizable (`Merci Boilerplate`). - Se actualizaron `README.md` e `instrucciones.md` para reflejar la nueva misión del repositorio. - Se refactorizó `public/index.html` para convertirlo en una página de presentación técnica del Boilerplate, explicando la arquitectura de «Núcleo Estático» y «Capa Dinámica». - Se añadió `LICENSE` (MIT) para formalizar la apertura del repositorio. - Se integró y eliminó la rama `feat/fase-3-diseno` mediante `git merge` y `git branch -d`, manteniendo el árbol de Git limpio.

Detalle técnico: La refactorización del `index.html` reemplazó el contenido personal por explicaciones de los componentes de la plantilla, reutilizando los mismos componentes BEM (`home-grid`, `home-card`) como demostración visual. La marca de autora (`mercedev.es`) se mantuvo incrustada por diseño en el footer, header y metadatos, diferenciando entre la identidad del proyecto y el contenido específico del sitio.

La eliminación de la rama de desarrollo sigue el principio de ciclo de vida corto para ramas de funcionalidad:

```
git checkout main
git merge feat/fase-3-diseno
git branch -d feat/fase-3-diseno
```

Motivo / criterio: Separación de responsabilidades a nivel macro (Arquitectura vs. Producto final). Construir un boilerplate permite abstraer y reutilizar las medidas de seguridad *Shift-Left* y las optimizaciones de rendimiento en múltiples webs futuras, maximizando el retorno de la inversión de tiempo de ingeniería. Las ramas de funcionalidad deben tener ciclos de vida cortos y eliminarse inmediatamente tras su integración para evitar repositorios con ramas «zombi».

Fuentes / Bibliografía: - Guía de flujo de trabajo Git (Git Flow): <https://nvie.com/posts/a-successful-git-branching-model>. - Licencia MIT: <https://opensource.org/licenses/MIT>.

2026-04-16 — Auditoría estática PHP y QA pre-despliegue (Fase 5.4)

Contexto: Con la introducción de WordPress y su `functions.php`, el auditor `merci-audit.py` debía ser capaz de detectar patrones de código PHP peligrosos que son vectores comunes de vulnerabilidades de ejecución remota de código (RCE — Remote Code Execution). Paralelamente, antes de iniciar la Fase 6 (despliegue a producción), era necesario ejecutar una auditoría integral sobre todo el repositorio para certificar que ninguna integración parcial había introducido regresiones o vulnerabilidades.

Hecho: - Se implementó la función `audit_php_smells` en `merci-audit.py`, con una expresión regular que detecta en archivos `.php` el uso de funciones de alto riesgo: `eval()`, `exec()`, `shell_exec()`, `system()`, `passthru()`, `popen()` y `proc_open()`. - Se clasificó el hallazgo como `WARN` (no como `ERROR`), permitiendo al desarrollador revisar el contexto manualmente antes de

decidir si es un uso legítimo o una vulnerabilidad real. - Se ejecutó la auditoría integral con `python3 scripts/merci/merci-audit.py --strict-json-ld` sobre todo el repositorio: resultado `0 ERROR, 0 WARN`. - Se creó el documento `docs/checklist-hardening.md` con todas las medidas de seguridad implementadas: directivas CSP, hooks de bloqueo de WordPress, política de permisos del servidor y reglas bloqueantes del auditor. - Se marcó la Fase 5.4 como completada en el `README.md`.

Detalle técnico: La función de auditoría PHP funciona mediante búsqueda de patrones en el contenido de los archivos, sin ejecutar el código:

```
DANGEROUS_PHP_FUNCTIONS = [
    'eval', 'exec', 'shell_exec', 'system',
    'passthru', 'popen', 'proc_open'
]

pattern = re.compile(
    r'\b(' + '|'.join(DANGEROUS_PHP_FUNCTIONS) + r')\s*\(',
    re.IGNORECASE
)
```

La distinción entre `ERROR` (código de salida `1`, bloquea el commit) y `WARN` (código de salida `0`, informativo) es fundamental para la experiencia de desarrollo: un plugin legítimo podría usar `exec()` para operaciones del sistema justificadas, y bloquearlo automáticamente crearía una fricción inaceptable.

Motivo / criterio: Seguridad *Shift-Left*. Al detectar el uso de estas funciones antes de que el código llegue al repositorio, se reduce drásticamente la probabilidad de introducir una puerta trasera accidentalmente, especialmente a través de código de terceros (plugins o temas de WordPress copiados sin revisión). Un *Sanity Check* pre-despliegue con código de salida `0` es la garantía empírica de que las prácticas de calidad se han mantenido desde la Fase 1.

Fuentes / Bibliografía: - OWASP Top 10 sobre inyección de código y RCE: https://owasp.org/Top10/A03_2021-Injection. - Documentación de PHP sobre funciones de ejecución de comandos del sistema. - Asistencia mediante IA para el

diseño de la estrategia de clasificación `WARN` vs `ERROR` en análisis de código PHP.

2026-04-17 — Inicio de Fase 6 y Deployment Playbook

Contexto: Con la auditoría local en verde y el Boilerplate consolidado, llegaba el momento de transicionar el proyecto desde el entorno de desarrollo local hacia la infraestructura de producción. Este paso es el más crítico del ciclo de vida de un proyecto web: un despliegue mal ejecutado puede exponer vulnerabilidades, corromper la base de datos o hacer inaccesible el sitio durante horas. La solución era documentar cada paso antes de tocar el servidor.

Hecho: - Se redactó el manual de operaciones `docs/deployment-playbook.md` antes de realizar cualquier acción en el servidor de producción. - El Playbook se estructuró en fases operativas: Fase 0 (DNS e Infraestructura), Fase 1 (Aprovisionamiento LEMP), Fase 2 (Clonación vía Git), Fase 3 (Aislamiento de WordPress con symlinks), Fase 4 (Enrutamiento Nginx + SSL) y Fase 5 (Verificación final). - Se sometió el repositorio a una auditoría externa automatizada (GitHub Copilot) que validó la arquitectura híbrida, la seguridad y el aislamiento DevSecOps con la máxima calificación. - Se fijaron las versiones de dependencias Python con `==` en lugar de `>=` en `requirements.txt` (ej. `Pillow==10.2.1`). - Se adoptó CloudPanel como panel de administración del servidor de producción por su compatibilidad nativa con Nginx, PHP-FPM y MariaDB, y por su gestión automática de renovación de certificados SSL.

Detalle técnico: Fijar versiones con `==` en gestores de paquetes es una práctica fundamental de reproducibilidad en DevOps. El operador `>=` expone el despliegue a *breaking changes* si se publica una versión mayor de la librería entre el desarrollo y la puesta en producción:

```
# ❌ No reproducible – puede instalar Pillow 11.x en producción
Pillow>=10.0.0
```

```
#  Reproducible – instala exactamente el binario auditado en local  
Pillow==10.2.1
```

CloudPanel gestiona sus propios bloques `server` en Nginx mediante plantillas con la variable `{{root}}`. Intentar sustituir esta variable manualmente por rutas absolutas en el editor de texto rompe la integración del panel ante cualquier actualización del sistema. La práctica correcta es modificar el Document Root desde la interfaz visual del panel (que propaga el cambio de forma segura a todas las variables `{{root}}`) y reservar la edición del bloque VHost para las reglas de enrutamiento personalizado.

Motivo / criterio: Reducción de riesgo operativo. Documentar el paso a paso («Runbook» o «Playbook») antes de tocar el servidor de producción previene errores por omisión, garantiza que se replican las políticas de seguridad estrictas y convierte el despliegue en una tarea rutinaria y auditable. Un Playbook sin ambigüedades es reproducible por cualquier miembro del equipo o en cualquier infraestructura futura.

Fuentes / Bibliografía: - Documentación oficial de CloudPanel: <https://www.cloudpanel.io/docs>. - Documentación de Let's Encrypt sobre el proceso de validación HTTP-01: <https://letsencrypt.org/docs/challenge-types>. - Asistencia mediante IA para la validación externa de la arquitectura (análisis de madurez pre-producción).

2026-04-17 — Resolución de obstáculos del despliegue en producción

Contexto: El despliegue real en el servidor de CloudPanel reveló una serie de problemas en cascada que no eran visibles en el entorno de desarrollo local. Cada uno de estos problemas tenía una causa técnica precisa relacionada con las diferencias entre un entorno local (directo, sin capas de abstracción) y un entorno de producción con proxies inversos, paneles de control y capas de caché.

Hecho: - Latencia inaceptable (290ms): Se diagnosticó que el servidor estaba provisionado en un datacenter de Asia/Oceanía. Se destruyó la máquina virtual y se provisionó una nueva en una región europea (Frankfurt/Ámsterdam). La latencia se redujo a valores normales. - **Error NXDOMAIN en SSL:** Let's Encrypt rechazó emitir el certificado porque el subdominio `www.mercedev.es` no tenía registro `A` en la zona DNS. Se emitió el certificado exclusivamente para el dominio raíz (`mercedev.es`). - **Mixed Content (HTTP/HTTPS):** WordPress generaba URLs `http://` porque `is_ssl()` de PHP devolvía `false` al no detectar el protocolo detrás del proxy Varnish de CloudPanel. Se resolvió sustituyendo `$_SERVER['HTTP_HOST']` por `home_url()` de WordPress, que lee la URL base desde la base de datos (ya configurada como `https://`). - **Puerto 8080 inyectado por Varnish:** Varnish añadía su puerto interno (`8080`) a la variable `$_SERVER['HTTP_HOST']`, generando URLs inválidas (`//mercedev.es:8080/css/main.css`). Se resolvió extrayendo la raíz del dominio con `preg_replace('#/blog/?$#', '', home_url())`. - **WooCommerce en modo «Coming Soon»:** Las versiones modernas de WooCommerce (`>= 9.0`) activan por defecto una pantalla de mantenimiento (`woocommerce_coming_soon`) en la base de datos tras su instalación, que secuestraba el enrutamiento e ignoraba el Child Theme. Se desactivó desde el panel de administración de WordPress.

Detalle técnico: La raíz del problema del Mixed Content y el puerto Varnish es la misma: leer variables de servidor crudas (`$_SERVER`) detrás de un proxy de alto rendimiento (Nginx + Varnish) es un antipatrón. El proxy termina la conexión TLS y reenvía la petición al PHP-FPM interno en HTTP plano, por lo que `is_ssl()` siempre devuelve `false`. La solución definitiva usa la abstracción nativa del framework:

```
// ❌ Antipatrón – lee variables crudas del servidor detrás de un proxy
$domain_root = (is_ssl() ? 'https://' : 'http://') .
$_SERVER['HTTP_HOST'];

// ✅ Correcto – usa la fuente de verdad de WordPress (base de datos)
$domain_root = preg_replace('#/blog/?$#', '', home_url());
```

La latencia de 290ms sin pérdida de paquetes es un síntoma inequívoco de distancia transcontinental causada por las limitaciones físicas de la fibra óptica, no de saturación de red local. La selección de la región geográfica del datacenter es

el primer paso innegociable de un despliegue: por mucho que se optimice el código, el TTFB (Time to First Byte — Tiempo hasta el Primer Byte) base está condicionado por la física.

Motivo / criterio: Resiliencia arquitectónica. Cada problema resuelto evidencia una lección estructural: la separación entre el estado de la base de datos y el código, el respeto por las abstracciones del framework, el conocimiento del ciclo de vida de las capas de proxy. Documentar estos obstáculos en la bitácora convierte cada error en un activo de conocimiento que previene los mismos problemas en futuros despliegues.

Fuentes / Bibliografía: - Documentación de WordPress sobre `home_url()` y la función `is_ssl()` : https://developer.wordpress.org/reference/functions/home_url. - Documentación de CloudPanel sobre la arquitectura interna de Nginx + Varnish: <https://www.cloudpanel.io/docs>. - Asistencia mediante IA para el diagnóstico del antipatrón de lectura de variables de servidor crudas detrás de proxies inversos.

2026-04-23 — Depuración CSP avanzada: scripts en línea de WooCommerce y whitelist SHA-256

Contexto: Tras resolver los problemas de despliegue, la auditoría de Google PageSpeed Insights en la ruta dinámica `/blog/tienda/` reveló violaciones de la CSP (Content Security Policy — Política de Seguridad de Contenidos) causadas por scripts en línea inyectados por WordPress y WooCommerce. Cada violación aparecía como un error en la consola del navegador y reducía la puntuación de «Mejores Prácticas» de 100 a 92. Con una directiva `script-src 'self'` estricta, cualquier `<script>...</script>` sin un hash autorizado es bloqueado.

Hecho: - Se identificaron y erradicaron las **Speculation Rules** de WordPress (JSON inyectado en `wp_head` y `wp_footer`) mediante `remove_action('wp_print_speculation_rules')` en ambos hooks. - Se eliminaron los **filtros SVG de Gutenberg** (`wp_global_styles_render_svg_filters`) inyectados en `wp_body_open` y `wp_footer`. - Se identificó el script `wc_javascript_is_active` de

WooCommerce (inyectado en `wp_head` con prioridad `0`) y se intentó eliminarlo con `remove_action('wp_head', 'wc_javascript_is_active', 0)`. - Se diagnosticó una **condición de carrera (Race Condition)**: el `remove_action` fallaba silenciosamente porque se ejecutaba antes de que WooCommerce registrara su hook. Se resolvió encapsulando todas las purgas en la función `merci_purgar_inyecciones_inline` anclada al hook `init`. - Cuando la condición de carrera persistió (WooCommerce usa prioridad `0`, que se ejecuta antes del ciclo normal), se aplicó la solución definitiva: **whitelist criptográfico SHA-256**. Se calculó el hash exacto del script (`sha256-eHL/Izx7K/qWL0kdBXXnHwsLSHvG0Jn/THLHydUZdog=`) y se añadió a la directiva `script-src` en la cabecera CSP de Nginx.

Detalle técnico: Una condición de carrera en el ciclo de vida de WordPress ocurre cuando un `remove_action` se ejecuta antes de que el plugin objetivo haya registrado su hook. La solución es envolver la purga en `init`, que se dispara cuando el core y todos los plugins ya están cargados en memoria:

```
// ❌ Condición de carrera – se ejecuta antes de que WooCommerce
registre su hook
remove_action('wp_head', 'wc_javascript_is_active', 0);

// ✅ Correcto – se ejecuta cuando todo está cargado
add_action('init', 'merci_purgar_inyecciones_inline');
function merci_purgar_inyecciones_inline() {
    remove_action('wp_head', 'wc_javascript_is_active', 0);
    remove_action('wp_head', 'wp_print_speculation_rules');
    remove_action('wp_footer', 'wp_print_speculation_rules');
    remove_action('wp_body_open',
'wp_global_styles_render_svg_filters');
    remove_action('wp_footer', 'wp_global_styles_render_svg_filters');
}
```

El whitelist criptográfico SHA-256 en la cabecera CSP de Nginx es la técnica más sofisticada del proyecto. Las versiones modernas de WooCommerce inyectan `wc_javascript_is_active` a nivel de renderizado de bloques (Gutenberg), haciéndolo invulnerable a los hooks tradicionales de PHP. En lugar de relajar la

seguridad permitiendo `'unsafe-inline'`, se autoriza exclusivamente el hash del script legítimo:

```
# En el bloque server de Nginx (VHost de CloudPanel)
add_header Content-Security-Policy "script-src 'self' 'sha256-eHL/
Izx7K/qWL0kdBXXnHwsLSHvG0Jn/THLHydUZdog=';" always;
```

Si un atacante modifica un solo carácter del script, el hash cambia y el navegador lo bloquea instantáneamente. Si el script es completamente estático (como este), el hash nunca cambia entre peticiones.

Motivo / criterio: Tolerancia cero ante la deuda técnica de seguridad. Ignorar un script bloqueado en la consola asumiendo que «es de un plugin y no tiene solución» es el primer paso hacia la degradación estructural de un proyecto. El whitelist criptográfico es la solución correcta en DevSecOps avanzado: cuando un script en línea benigno y estático no puede ser erradicado del código legado, la solución no es debilitar la política de seguridad, sino autorizar quirúrgicamente la huella criptográfica exacta de ese script concreto.

Fuentes / Bibliografía: - Especificación del W3C sobre hashes en Content Security Policy: <https://www.w3.org/TR/CSP3/#source-hash>. - Documentación de MDN sobre el ciclo de vida de hooks en WordPress y condiciones de carrera. - Asistencia mediante IA para el diagnóstico de la condición de carrera con la prioridad 0 de WooCommerce y para el cálculo del hash SHA-256 del script en línea.

2026-04-23 — Milestone: Cierre de Fase 6 y validación 100/100 en PageSpeed

Contexto: Tras resolver todos los conflictos de CSP y los problemas de despliegue en producción, era el momento de la validación empírica final. El objetivo declarado del proyecto era demostrar que es posible construir una web híbrida (núcleo estático + WordPress + WooCommerce) que cumpla con los más

altos estándares de la industria en rendimiento, seguridad, accesibilidad y SEO técnico simultáneamente.

Hecho: - Se validó una puntuación perfecta (100/100) en todas las categorías de Google PageSpeed Insights para la ruta dinámica `/blog/tienda/` (WooCommerce), tanto en vista móvil como de escritorio. - Se confirmó la ausencia total de errores de CSP o JavaScript en la consola del navegador. - Se ejecutó una auditoría manual de accesibilidad: navegación completa solo con teclado (tabulación), verificación de estados de foco, flujo visual vs. DOM, comportamiento del menú fuera de pantalla y revisión de Landmarks semánticos y etiquetas `aria-label`. - Se inyectó la función `merci_inyectar_metadatos_seo` en `functions.php`, anclada al hook `wp_head`, para generar dinámicamente la etiqueta `<meta name="description">` en las páginas de WordPress (ausente por defecto en WordPress nativo). - Se actualizó `docs/checklist-hardening.md` con la fecha de revisión final. - Se marcó el cierre definitivo de la Fase 6 en el `README.md`.

Detalle técnico: Conseguir 100/100 en Core Web Vitals dentro de un ecosistema WooCommerce es atípico. Los resultados validan empíricamente la arquitectura de «doble escudo»: el **escudo de rendimiento** en `functions.php` (desencolado de scripts, bloqueo de hooks, atributo `defer` global en JavaScript) y el **escudo de infraestructura** en Nginx (cabecera CSP con hashes criptográficos, HSTS, COOP).

La micro-corrección más reveladora fue la del CLS (Cumulative Layout Shift — Desplazamiento Acumulativo del Diseño): el atributo `height="auto"` en el logotipo es inválido en HTML5 y provoca que el navegador no reserve espacio vertical antes de cargar la imagen, causando un micro-salto visual. La corrección fue especificar las dimensiones exactas (`width="263" height="65"`), que eliminó completamente el CLS.

La función de meta descripción dinámica evalúa el contexto de WordPress (`is_shop()`, `is_category()`, `is_singular()`) para extraer el texto apropiado, e incluye una validación `class_exists('WPSEO_Meta')` para desactivarse automáticamente si se instala un plugin de SEO especializado en el futuro, evitando etiquetas duplicadas.

La auditoría manual de accesibilidad cerró la brecha entre la métrica técnica y la empatía con el usuario final: un 100/100 en herramientas automatizadas no garantiza que un usuario con tecnologías de asistencia pueda navegar lógicamente por la página.

Motivo / criterio: La consecución de este hito valida empíricamente la tesis del proyecto: es posible utilizar un CMS pesado para la gestión de contenidos sin sacrificar en absoluto la velocidad de carga, la seguridad ni la experiencia de usuario. La separación de responsabilidades y el enfoque *Shift-Left* en rendimiento y seguridad no son restricciones que limiten las funcionalidades, sino la base sobre la que se construyen proyectos web de alta calidad.

Fuentes / Bibliografía: - Google PageSpeed Insights: <https://pagespeed.web.dev>. - Documentación de Google sobre Core Web Vitals (LCP, INP, CLS): <https://web.dev/vitals>. - Especificación WAI-ARIA del W3C sobre auditorías manuales de accesibilidad: <https://www.w3.org/TR/wai-aria>. - Asistencia mediante IA para el análisis de la arquitectura de «doble escudo» y la justificación de las decisiones de pragmatismo (Trusted Types, CSS bloqueante).

2026-04-21 — Orquestador maestro del pipeline: merci-total

Contexto: Con el ecosistema completo de herramientas del Sistema Merci funcionando (optimizador, compilador SASS, sitemap, auditor estático, rastreador de enlaces), ejecutarlas individualmente antes de cada pase a producción generaba fricción operativa y riesgo real de omitir pasos críticos. Era necesario un único comando que garantizara que el código siempre se optimiza, compila y audita antes de integrarse.

Hecho: - Se creó `scripts/merci/merci-total.py`, el orquestador maestro que ejecuta secuencialmente toda la cadena de herramientas del proyecto. - Se implementó el patrón **Fail-Fast**: si cualquier subprocesso falla (devuelve código de salida distinto de `0`), la ejecución se detiene inmediatamente. - Se excluyeron explícitamente del orquestador los procesos interactivos (`merci-commit.py`) y

los demonios (`merci-watcher.py`). - Se inyectó el alias `merci-total` en `~/.zshrc` .

Detalle técnico: El pipeline lógico del orquestador sigue este orden:

```
merci-optimizer.py  →  Optimización de assets (WebP, variantes  
responsivas)  
merci-styles.py     →  Compilación SASS → CSS  
merci-sitemap.py    →  Actualización de fechas en sitemap.xml  
merci-audit.py      →  SAST (Static Application Security Testing)  
merci-linkcheck.py  →  DAST (Dynamic Application Security Testing –  
rastreo HTTP)
```

El patrón Fail-Fast se implementa verificando el código de retorno de cada subprocesso mediante `subprocess.run(..., check=True)` , que lanza una excepción `CalledProcessError` ante cualquier fallo, deteniendo la cadena en el punto exacto del error y mostrando al desarrollador cuál herramienta falló.

Motivo / criterio: CI/CD (Continuous Integration / Continuous Deployment — Integración Continua / Despliegue Continuo) local. Consolidar toda la cadena de suministro en un único comando garantiza que el código siempre esté optimizado y auditado antes de integrarse, independientemente de la disciplina individual del desarrollador. El patrón Fail-Fast evita que un error silencioso en una etapa temprana contamine etapas posteriores con datos incorrectos.

Fuentes / Bibliografía: - Documentación oficial de Python sobre el módulo `subprocess` y el parámetro `check=True` . - Asistencia mediante IA para el diseño del pipeline secuencial y la estrategia de exclusión de procesos interactivos y demonios. - Principios de CI/CD: <https://martinfowler.com/articles/continuousIntegration.html>.